



DELIVERY APP UPDATE · APRIL 3

Feature 5: Admin & Agent Dashboard

Bhavya

Feature Owner

3rd April 2026

Table of Contents

1. What We Did Today: Feature 5 Complete.....	3
2. File 1: DeliveryRequestRepository.java — One Addition.....	3
3. File 2: DashboardService.java — The Aggregation Service.....	3
4. File 3: DashboardController.java — Three Endpoints.....	4
5. File 4: admin-dashboard.html — The Admin View.....	5
6. File 5: agent-dashboard.html — The Agent View.....	5
7. Feature 5: Confirmed Complete.....	6
8. What's Next.....	6

What We Did Today: Feature 5 Complete

Today's session built Feature 5 end to end in a single sitting — all five new files written and committed to the **feature-dashboard** branch. Feature 5 gives two roles their own dashboard: the Admin gets a live overview of every order in the system, and the Agent gets a personal view of only their own work. Below is a walk-through of every file added today and exactly what it does.

The commit was staged in a local working clone because the mounted filesystem had a stale git lock file. The commit hash is **790252d**. To push, run `git push origin feature-dashboard` from IntelliJ or your terminal — the push requires your macOS keychain credentials.

File 1: DeliveryRequestRepository.java — One Addition

This repository already existed from Feature 4. One new query method was added to support the dashboard's two-panel split: **findByIsImmediate(boolean isImmediate)**.

What was added

The method lets the dashboard retrieve all immediate orders as one group and all scheduled orders as another, without filtering by status. This is what powers the two side-by-side panels on the admin dashboard. Spring Data generates the SQL automatically from the method name: `SELECT * FROM delivery_requests WHERE is_immediate = ?`

Code added

```
// Get all orders of a given type - immediate (Order Now) or scheduled
// Used by the admin dashboard to populate the two side-by-side panels
List<DeliveryRequest> findByIsImmediate(boolean isImmediate);
```

File 2: DashboardService.java — The Aggregation Service

DashboardService is the business logic layer for Feature 5. It is injected with both **DeliveryRequestRepository** and **AgentRepository**, pulls data from both, and assembles it into exactly what the dashboard pages need. Neither repository knows about the other — DashboardService is the one combining them. This is the **AGGREGATION pattern** — a key OOP concept worth mentioning in your viva.

Admin Dashboard methods

- **getLiveOrders()** — returns all immediate orders not yet in a terminal state (DELIVERED or FAILED). These are the “Order Now” deliveries that need immediate attention. Uses Java Streams to filter in memory.
- **getScheduledOrders()** — returns all time-slot-based orders still active, sorted by time slot date so the earliest delivery appears first.
- **countTotalActive()** — counts all orders not in a terminal state. Shown as “Total Active” in the summary bar at the top of the admin dashboard.

- **countDeliveredToday()** — counts DELIVERED orders where updatedAt matches today's date. The setStatus() method in DeliveryRequest always refreshes updatedAt, so this timestamp is reliable.
- **countFailedToday()** — same pattern as countDeliveredToday but for FAILED orders.
- **countPendingAssignment()** — counts PLACED orders (no agent assigned yet). Tells the admin how many orders still need actioning.
- **getAllAgents()** and **getAvailableAgents()** — used to populate the filter dropdown and the assign-agent form.

Filter method

filterOrders(String status, Boolean isImmediate, Long agentId) — all three parameters are optional (null means no filter on that dimension). Uses a Java Stream chain to apply each condition independently. If the admin leaves a filter blank, that filter simply does not apply.

Agent Dashboard methods

- **getAgentCurrentOrders(Agent)** — returns orders with status ASSIGNED or OUT_FOR_DELIVERY for the given agent. These are the deliveries the agent is actively working on right now.
- **getAgentScheduledOrders(Agent)** — returns SCHEDULED orders assigned to the agent, sorted by time slot date. These are upcoming deliveries the agent has not started yet.

Assign method

assignAgentToOrder(Long orderId, Long agentId) — fetches both the order and the agent by ID, sets the agent on the order, and if the order is still in PLACED status advances it to ASSIGNED. Also increments the agent's currentDeliveryCount. Both the order and the agent are then saved back to the database.

File 3: DashboardController.java — Three Endpoints

DashboardController is the web layer — it receives HTTP requests, asks DashboardService for data, puts the data into the Thymeleaf Model, and returns the template name. It has no business logic of its own. Three endpoints:

GET /admin/dashboard — Admin view (with optional filter params)

Checks that an ADMIN is logged in via session. Reads three optional @RequestParam values — status, isImmediate, agentId — which are all null if the admin has not filtered yet. If any filter is set, calls **filterOrders()** and splits the result into live and scheduled lists. If no filter, calls **getLiveOrders()** and **getScheduledOrders()** directly. Always computes the four summary counts unfiltered so the numbers at the top always reflect the real system state. Returns “admin-dashboard”.

POST /admin/assign — Agent assignment

Receives orderId and agentId as form fields from the assign form on the dashboard. Calls **assignAgentToOrder()** on the service. Uses POST-Redirect-GET: after the assignment, always redirects back to /admin/dashboard with a flash message. This prevents accidental re-submission if the admin refreshes the page.

GET /agent/dashboard — Agent personal view

Checks that an AGENT is logged in. Fetches their Agent profile via **agentRepository.findByUser(loggedInUser)** — note that the User object (login identity) is different from the Agent object (operational profile). If no Agent profile is found, shows a graceful error message rather than crashing. Otherwise calls **getAgentCurrentOrders()** and **getAgentScheduledOrders()** and returns “agent-dashboard”.

File 4: admin-dashboard.html — The Admin View

A Bootstrap 5 Thymeleaf template that renders the full admin dashboard. Four parts:

- **Summary cards row:** Four Bootstrap cards — Total Active (blue), Delivered Today (green), Failed Today (red), Pending Assignment (yellow). Each card shows a large number using th:text. These always show system-wide totals, never the filtered count.
- **Filter bar:** A GET form with three dropdowns — status (all DeliveryStatusEnum values), type (Immediate / Scheduled), and agent (all agents by name). Submitting appends params to the URL so the URL always reflects what is currently filtered. A “Clear” link goes back to the plain URL. A “Filter active” badge appears when any filter is set.
- **Two panels:** Left panel (dark header) shows Live Orders; right panel (grey header) shows Scheduled Orders. Each is a Bootstrap card with a table inside. The scheduled panel has an extra Slot column showing the time slot date and label. Both panels show an empty-state message when no orders are present.
- **Assign form (inline):** If an order has no agent assigned, an inline dropdown + Assign button appears in the Agent column. Submitting POSTs to /admin/assign. If an agent is already assigned, their name is shown instead. Status and priority badges are colour-coded using Thymeleaf nested ternary expressions.

File 5: agent-dashboard.html — The Agent View

A Bootstrap 5 Thymeleaf template for the agent's personal dashboard. Intentionally lean — the agent only sees their own orders. Two sections:

- **Right Now:** Orders with status ASSIGNED or OUT_FOR_DELIVERY assigned to the logged-in agent. Each card shows the pickup and delivery addresses, customer name and phone, any special instructions, the priority badge, and an “Update Status →” button linking to the tracking page so the agent can advance the order status.
- **Today's Schedule:** SCHEDULED orders assigned to this agent, sorted chronologically by time slot date. Each card shows the time slot date and label in a badge, plus all the same order details. A “View Order →” button links to the tracking page.
- At the top of the page, an availability badge (green = Available, grey = Unavailable) and a delivery count badge give the agent a quick at-a-glance status check. If the agent account has no Agent profile linked to it, a graceful error message is shown rather than a crash.

Feature 5: Confirmed Complete

All files that make up Feature 5 are written and committed. The full list:

- **DeliveryRequestRepository.java** — one new method: `findByIsImmediate()`
- **DashboardService.java** — full aggregation service: admin data, agent data, filter logic, summary counts, assign-agent operation
- **DashboardController.java** — three endpoints: GET `/admin/dashboard`, GET `/agent/dashboard`, POST `/admin/assign`
- **admin-dashboard.html** — Bootstrap 5 Thymeleaf template with summary cards, filter bar, two-panel order view, inline assign forms
- **agent-dashboard.html** — Bootstrap 5 Thymeleaf template with Right Now section and Today's Schedule section

Commit hash: **790252d** on branch **feature-dashboard**. Push from IntelliJ or terminal: `git push origin feature-dashboard`

Key OOP concept for viva: DashboardService uses the AGGREGATION pattern — it holds references to multiple repositories (DeliveryRequest, Agent) and combines data from both into a single assembled view. This is the Service Layer pattern: all cross-repository logic lives in the service, not in controllers or repositories.

What's Next

Both Bhavya's features (4 and 5) are now fully complete. The remaining steps are:

Date	Task
April 3 (today)	Push feature-dashboard branch to GitHub from IntelliJ <code>git push origin feature-dashboard</code>
April 10–12	Integration, polish, and testing across all branches
End of project	Bhavya designs and builds the full UI across all pages